Remote Controller with Fail-Safe Update

Software Classification

Reference	TBD **[TBD-1]**
Issue / revision	Issue 1, Revision 0 — DRAFT
Date	10 October 2026
Author	Andrew Toma
Open items	2 TBD (App. A)

Document change record

Issue	Rev.	Date	Change
1	0	2026-10-10	First draft

1.0 Purpose and Scope

This document establishes the software classification of each software item, and so the rigour each item requires. It is the formal record of the software classification determinations [1, Sec. 3.5.2, p. 25].

The evaluated software items encompass the System software and its enabling systems, as defined in the Concept of Operations [4, Sec. 1.2.2]. The System models an uncrewed spacecraft whose primary mission objectives directly depend on this software; a failure would cost those objectives.

2.0 Documents

[1] NASA, *NASA Software Engineering Requirements*, NPR 7150.2D.
[2] NASA, *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, 2022.
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, page "SWE-020 – Software Classification".
[4] *Remote Controller with Fail-Safe Update — Concept of Operations*, Issue 1, Revision 0, Draft.

3.0 Classification

Each item takes the highest class whose definition applies [1, Sec. 3.5.1, p. 24].

Software item	Role	Class
Controller firmware	FLIGHT	B
Onboard computer software	FLIGHT	B
Ground software in the mission operations center	GROUND	B
Bench test tools	TEST	C
Firmware build tools	BUILD	D

Legend: role

FLIGHT	Operates the spacecraft from orbit.
GROUND	Operates the spacecraft from the ground.
TEST	Tests the System; never operates the spacecraft.
BUILD	Builds the System; never operates the spacecraft.
Class	As defined in [1, App. D, pp. 79–86].

3.1 Rationale

Controller firmware: Class B. Class B covers flight software "required to operate the vehicle or space asset … such as commanding of the vehicle or asset" [1, App. D, p. 81]; its examples include "fault protection" [1, App. D, p. 81]. The controller runs commands and returns to working firmware on its own when new firmware fails.

Onboard computer software: Class B. The same definition applies. It passes every command and firmware image to the controller, watches it, and restores it when its firmware cannot run.

Ground software: Class B. Class B examples include "command and control ground systems" [1, App. D, p. 81]. The ground software prepares and checks every command and firmware image sent to the spacecraft.

Bench test tools: Class C. Class C covers "software used for the testing of space assets" [1, App. D, p. 82], and Class B excludes systems "used in testing Class B systems" in a development environment [1, App. D, p. 81].

Firmware build tools: Class D. Class D covers "ground software tools that support engineering development" [1, App. D, p. 83].

Why not Class A. Class A is software for human space flight [1, App. D, p. 79]. The imitated spacecraft carries no crew.

3.2 What Each Class Requires

The class sets which requirements of NPR 7150.2D apply: those marked for that class in its Requirements Mapping Matrix [1, Sec. 3.1.11, p. 22]. Lower classes carry fewer requirements and less rigour [1, App. D, p. 79].

Class	Requirements that apply	Items
B	100 of 100	Controller, onboard computer and ground software
C	92 of 100	Bench test tools
D	64 of 100	Firmware build tools

Counted from [1, App. C, pp. 56–78]. Class B adds eight requirements to Class C, identified by NASA's own numbers:

SWE-141 Software independent verification and validation (IV&V) for the project categories listed in [1, Sec. 3.6.2, p. 62].

SWE-131 "If software IV&V is performed", an IV&V plan is developed and executed [1, Sec. 3.6.3, p. 63].

SWE-178 "If software IV&V is performed", IV&V gets access to all development artifacts [1, Sec. 3.6.4, p. 63].

SWE-179 "If software IV&V is performed", its issues and risks are answered and tracked to closure [1, Sec. 3.6.5, p. 63].

SWE-032 Software from an organization with a "CMMI-DEV Maturity Level 2 Rating or higher" for Class B [1, Sec. 3.9.2, p. 66].

SWE-193 "Acceptance tests for loaded or uplinked data, rules, and code that affects software and software system behavior" [1, Sec. 4.5.13, p. 73].

SWE-200 Software requirements volatility metrics, collected and reported [1, Sec. 5.4.6, p. 77].

SWE-204 "Process assessments for all high severity software non-conformances" [1, Sec. 5.5.4, p. 78].

4.0 Safety-Critical Designation

Software is safety-critical when a hazard analysis traces a hazard to it [2, Sec. 4.2, p. 20]; the determination is made for each component [1, Sec. 3.7.1, p. 26]. The designation of each item above is set after the hazard analysis **[TBD-2]**.

5.0 Tailoring and Review

- Software engineering and software assurance are expected to agree on each classification [1, Sec. 3.5.1 Note, p. 25]. One engineer builds the System, with no independent assurance team (ConOps constraint C-3), so the classification is the author's alone.
- The classification is revisited at each later phase [3, Sec. 3.5.1]. If a class changes, the plans are updated to match [1, Sec. 2.2.8, p. 19].

A Open Items

The values below are not yet known.

TBD-1 Project name and document reference.

TBD-2 Safety-critical designation of each software item, from the hazard analysis.